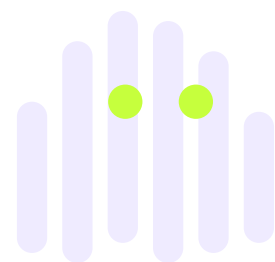


BUILD THE APP. EVERY SCREEN. SPECIFIED.

The complete product requirements for the GhostShopper platform: customer dashboard, AI calling pipeline, scoring engine, Stripe billing, multi-voice management and the internal admin area. Written to be followed step by step. Read pages 2 and 3 first, then build in the milestone order in section 9. Do not skip the safety requirements in section 8 under any circumstances.



SECTION 1

What GhostShopper is, in plain English

GhostShopper is AI mystery shopping for phone calls. Our customers are brands that run many locations: franchises, dental groups, home care networks, estate agencies. Every location answers its own phone, and a badly handled call loses the brand real money (a plumbing enquiry is a £150 to £2,000 job, a care enquiry can be a £40,000 a year client).

Our AI voice agents phone each of the customer's locations on a rolling schedule, pretending to be an ordinary customer with a realistic enquiry. The staff member who answers does not know the call is a test. We record the call, score it against the brand's own standards using an AI grader, and show head office a weekly league table of every location, with the evidence behind every point lost: the exact quote and the exact timestamp.

The core loop you are building: **schedule a call → AI agent makes the call → recording and transcript come back → AI grader scores it with evidence → customer sees it on a dashboard → billing runs monthly per location via Stripe**. Everything else in this document supports that loop.

The customer never talks to the AI. The customer sets standards and reads results. The AI talks to their staff, secretly, with the brand's signed authorisation.

Glossary you must know

- **Org**: one customer account (one brand). All data is scoped to an org.
- **Location**: one branch or franchise site with its own phone number.
- **Scenario**: the fictional customer story an AI caller performs (persona, enquiry, goals).
- **Scorecard**: the list of weighted criteria a call is graded against.
- **Shop / test call**: one AI mystery-shop call to one location.
- **Critical fail**: an automatic red flag, for example the staff never said the company name.
- **Attestation**: the signed authorisation from the brand confirming we may test its locations. Nothing may be called without it. This is a legal requirement, not a feature.
- **Review queue**: calls where the AI grader was unsure, waiting for a human to confirm the score.

SECTION 2

System overview and stack

The platform has four parts. The marketing site already exists and is not your job beyond linking to the app. You are building parts 2, 3 and 4.

Part	What it is	Who uses it
1. Marketing site	Static landing page with a qualifying lead form. Already built and deployed.	Prospects
2. Customer app	Dashboard where brands see league tables, calls, scores and evidence, manage locations, scorecards and billing.	Customer staff (owner, admin, coach roles)
3. Calling and scoring pipeline	Background services: scheduler, ElevenLabs call integration, webhook intake, Claude grading, review queue.	Nobody directly; it feeds the app
4. Admin area	Internal back office at /admin: manage every org, every call, voices, billing state, safety controls and the review queue.	Us (superadmin, operator roles)

Required stack (do not substitute without asking)

- **Next.js 14+ with TypeScript and App Router**, deployed on Vercel. One repo, /app for product, /site for the existing landing page.
- **Supabase**: Postgres, Auth (magic link email), Storage (call audio), Row Level Security on every table.
- **Tailwind CSS** with the brand tokens from CLAUDE.md (Ink #0F0C1D, Paper #FFFFFF, Ecto #C6FF3D, Lilac #EFEBFF, Violet #6C4CF1, Red #FF4D5E). Fonts: Archivo Black for headings, Inter for UI, Anton for hero lines only.
- **ElevenLabs Agents + Twilio** native integration for outbound calls (the agent and phone number already exist in our ElevenLabs account).
- **Anthropic API** (model claude-sonnet-4-6) for call grading and scenario generation, using JSON-only outputs.
- **Stripe**: Billing with per-location quantity pricing, Checkout, Customer Portal, webhooks.
- **Inngest or Vercel Cron** for scheduled jobs and the scoring queue. Resend for transactional email.

What already exists and must be reused, not rebuilt

- CLAUDE.md: brand system, copy rules, integrity rules. Read it before writing any UI.
- ghostshopper-landing.html: the live marketing site and the form lead shape.
- ghostshopper-scorer.html: a working prototype of the grading prompt and report UI. Your scoring service should produce the same JSON shape it uses.
- The Sarah scenario prompt and two scored Heatplex test transcripts: use as seed data and grader test fixtures.
- Copy rules that apply to every string you ship: British English, never use em dashes, GhostShopper is the noun and ghostshop is the verb.

SECTION 3

Users, roles and permissions

Role	Where	Can do	Cannot do
Owner	Customer app	Everything in their org: billing, users, locations, scorecards, disputes	See any other org
Admin	Customer app	Everything except billing and deleting the org	Billing changes
Coach	Customer app	View dashboards, calls, evidence; add coaching notes; raise disputes	Edit scorecards, locations, users
Location viewer	Customer app	View one assigned location's scores and calls only	See the network league table
Operator	Admin area	Review queue grading, call monitoring, voice pool management	Billing overrides, org deletion, safety switches
Superadmin	Admin area	Everything, including impersonation, safety switches and billing overrides	Nothing

Enforce roles in three layers: Supabase RLS policies on every table (org_id scoping plus role checks), server side checks in every API route, and UI hiding as the last cosmetic layer. Never rely on UI hiding alone. The admin area lives behind a separate allowlist of our email addresses plus a required second factor.

SECTION 4

Database schema

Create these as Supabase migrations. Every table has id (uuid), created_at, updated_at. Every org-owned table has org_id with an RLS policy. Field types are indicative; use sensible Postgres types.

```

orgs(name, plan, stripe_customer_id, subscription_status, attestation_signed_at,
attestation_signed_by, attestation_doc_url, settings_json)
users(org_id, email, role, assigned_location_id nullable)
locations(org_id, name, phone_e164, timezone, jurisdiction, recording_mode [record |
transcript_only | paused], call_window_json, frequency [weekly | fortnightly | monthly], status,
opt_out bool)
scorecards(org_id, name, version, published bool)
scorecard_criteria(scorecard_id, label, description, weight, kind [binary | scale |
critical_fail], sort)
scenarios(org_id, name, prompt_brief, persona_json, goals_json, difficulty, active bool)
voices(elevenlabs_voice_id, label, gender, accent, active bool, notes) [global, admin managed]
agent_profiles(scenario_id, voice_id, elevenlabs_agent_id) [one ElevenLabs agent per
scenario+voice pair]
shop_schedules(location_id, scenario_id, scorecard_id, next_run_at, cadence, last_voice_id)
calls(org_id, location_id, scenario_id, voice_id, scheduled_for, started_at, duration_s, status
[queued | dialing | no_answer | voicemail | completed | failed | opted_out], provider_call_id,
audio_path, transcript_json, retention_expires_at)
scores(call_id, scorecard_id, total, grader_model, graded_at, human_reviewed bool, flagged bool,
flag_reason)
score_items(score_id, criterion_id, value, max, evidence_quote, transcript_offset, confidence,
note)
disputes(score_id, raised_by, reason, status [open | upheld | amended], amended_total nullable,
notes)
subscriptions(org_id, stripe_subscription_id, tier, location_quantity, current_period_end,
status)
alerts(org_id, location_id, type [critical_fail | no_answer_streak | score_drop], payload_json,
sent_at)
audit_log(org_id nullable, actor, action, entity_type, entity_id, meta_json) [append only, admin

```

actions always logged]

Storage: call audio lives in a private Supabase bucket at calls/{org_id}/{call_id}.mp3, served only through short-lived signed URLs. A nightly job deletes audio past retention_expires_at (default 90 days). Scores and transcripts are kept.

SECTION 5

The customer app, screen by screen

Dark dashboard surfaces (Ink background) matching the league table styling on the landing page. Light surfaces for settings and forms. Every screen below lists its acceptance criteria; a screen is done when every criterion passes.

5.1 Sign in and onboarding

- Magic link email login via Supabase Auth. No passwords anywhere.
- First login creates the org: brand name, sector, then the attestation step: full legal text displayed, typed signature (name + role), checkbox confirming authority over the locations, timestamp and IP stored on the org.
- Acceptance: a new user reaches an empty dashboard in under two minutes; an org with no signed attestation shows a blocking banner and cannot create schedules or trigger calls, enforced server side.

5.2 Dashboard: the network league table

- Ranked list of locations for a selected week: rank, name, score bar (Ecto for pass, Red under 50 or no answer), score number, trend arrow versus previous week.
- Week selector, sector filter if the org tags locations, and three headline stats: network average, best mover, locations with critical fails.
- Clicking a row opens Location detail. Acceptance: renders the seeded demo org (Richmond 94 down to Sutton 0) exactly like the landing page league table, loads under one second with 200 locations.

5.3 Location detail

- Score trend chart across weeks, list of that location's calls with status chips, current schedule summary (scenario, cadence, next call date hidden from coach role to preserve test integrity: only owner and admin see exact next_run_at).
- Acceptance: a coach can get from league table to a specific call's evidence in two clicks.

5.4 Call detail: the evidence screen

- Audio player streaming from a signed URL. Transcript rendered with Staff and Caller turns and timestamps.
- Score panel: total, per-criterion rows with bars, each showing the evidence quote and a Listen at m:ss link that seeks the audio player to that offset.
- Critical fail and suspected AI detection banners when flagged. Dispute button opens a reason form and creates a dispute record.
- Acceptance: pressing Listen at 1:42 on the seeded Croydon call seeks the player to 1:42. This exact interaction is the product's signature; it must feel instant.

5.5 Scorecard editor

- List, create, edit criteria: label, description (this text is fed verbatim to the grader, say so in the UI), weight, kind. Weights must total 100; block publishing otherwise.
- Editing a published scorecard creates a new version; old scores keep pointing at their original version.
- Acceptance: version integrity proven by a test: rescoreing never mutates historic score_items.

5.6 Scenarios

- Customer writes a two-sentence brief. A Generate button calls Claude to expand it into persona_json and goals_json following the structure of the existing Sarah prompt (wait for greeting, goal ladder, privacy rules, AI-question deflection, polite exit). Show the generated persona for review; owner must approve before it becomes active.

- Acceptance: generated scenarios always include the safety rules block; a unit test asserts its presence.

5.7 Locations manager, team, and settings

- Locations: table CRUD plus CSV import with a mapping step and E.164 phone validation. Per-location frequency, call window, opt out toggle.
- Team: invite by email with role. Location viewers get exactly one assigned location.
- Settings: org profile, notification preferences, data retention choice (30/90/180 days audio).

SECTION 6

The calling and scoring pipeline

6.1 Voice pool (multiple voices)

- The voices table is a global pool of ElevenLabs voice IDs managed in the admin area (target: 6 to 10 UK voices, mixed gender and accent).
- For each active scenario+voice pair, create an ElevenLabs agent via their API and store it in agent_profiles, so changing a scenario updates all its agents.
- Rotation rule: when scheduling a call, pick a voice at random excluding the location's last_voice_id, so the same branch never hears the same voice twice in a row. Store the choice on the call record.

6.2 Scheduler

- A cron job every 15 minutes: find shop_schedules due (next_run_at passed), where the org attestation is signed, subscription status is active or trialing, the location is not opted out, and now is inside the location's call window in its own timezone.
- Create a queued call record, pick the voice, trigger the ElevenLabs outbound call API with the agent_profile and the location's phone number, then advance next_run_at by the cadence.
- No answer: retry up to two times at 90 minute gaps within the window; after that the call is finalised as no_answer, which scores 0 and is itself a finding.
- Frequency cap: never more than one test call per location per 5 days regardless of configuration errors.

6.3 Call intake webhook

- Endpoint receiving ElevenLabs post-call webhooks: verify the signature, match provider_call_id to our call record, store status, duration, transcript_json; download the recording to Supabase storage and set audio_path.
- Completed calls enqueue a scoring job. Voicemail and failed calls finalise without scoring.

6.4 Grading service

- Input: transcript plus the schedule's scorecard criteria. Call the Anthropic API (claude-sonnet-4-6) with a JSON-only grading prompt identical in shape to ghostshopper-scorer.html: total, verdict, critical_fail, suspected_ai_detection, per-criterion score, evidence_quote, timestamp, confidence, note, coaching_summary.
- Store into scores and score_items. Flag for human review when: any criterion confidence is below 0.7, critical_fail is true, suspected_ai_detection is true, or JSON parsing needed a retry.
- Weekly aggregates: a job computes each location's weekly score (mean of completed calls, no_answer counts as 0) into a materialised view the league table reads.
- Alerts: critical fails and three consecutive no answers create an alert record and send an email to org admins via Resend.

SECTION 7

Stripe billing

Tier	Price	Includes
Pilot audit	£499 one-off (Stripe Checkout payment)	10 locations tested once, full report. The sales team converts this to a subscription manually.
Growth	£499 per month	Up to 15 locations, weekly shops
Scale	£1,199 per month	Up to 40 locations, weekly shops, coaching seats

Tier	Price	Includes
Brand	£30 per location per month, minimum 41 locations, annual invoice option	Unlimited scale, API access later

- Implementation: Stripe Products and Prices created via a seed script, not by hand. Growth and Scale are flat monthly prices; Brand uses per-unit quantity equal to active location count, synced nightly and on every location change.
- Checkout: from the app's Billing page via Stripe Checkout; success webhook sets subscriptions and orgs.subscription_status. Customer Portal link for card changes, invoices and cancellation.
- Webhooks to handle: checkout.session.completed, customer.subscription.updated and .deleted, invoice.payment_failed (set status past_due, show an app banner, pause new call scheduling after 7 days past due; never delete data).
- Gating: scheduling requires active or trialing status. Dashboards remain readable when past due, because locked out customers never come back.
- A one-off £750 setup fee is added to the first invoice of any subscription via an invoice item.
- Acceptance: full lifecycle proven in Stripe test mode: subscribe, upgrade Growth to Scale, exceed the location cap (app blocks adding location 16 on Growth with an upgrade prompt), fail a payment, recover, cancel.

SECTION 8

Admin area and the non-negotiable safety rules

8.1 Admin screens (/admin, staff only)

- **Orgs:** list with subscription status, attestation status, location count, last activity. Org detail shows everything, with a View as customer read-only impersonation mode (banner always visible, every impersonation written to audit_log).
- **Calls monitor:** live table of today's calls across all orgs with status, filterable; ability to cancel a queued call and to replay any webhook.
- **Review queue:** flagged calls with audio, transcript and the AI's draft score side by side; operator confirms or corrects each criterion; corrections stored separately from model output (they become the eval set).
- **Voices:** manage the voice pool, preview each voice, activate or retire, see per-voice suspected-AI detection rates.
- **Scenarios library:** view any org's scenarios, edit the shared safety rules block that is injected into all of them.
- **Billing:** read-only Stripe mirror per org with deep links into the Stripe dashboard; comp and trial extension controls (superadmin only).
- **Safety switches:** see 8.2.
- **Audit log:** searchable, append only, every admin action recorded.

8.2 Safety rules. Build these first, test them hardest.

- **Attestation gate:** the scheduler and every call-triggering code path must verify orgs.attestation_signed_at is set. This check lives in one shared function used everywhere, with a test proving calls cannot be created without it.
- **Global allowlist:** environment variable CALL_ALLOWLIST containing phone numbers. While it is non-empty, outbound calls to any number not on it are refused at the final dial function, regardless of any other state. This ships ON with only our own numbers and is removed only by us, in production, deliberately.
- **Kill switch:** a single admin toggle that halts all outbound calling platform-wide within one scheduler cycle.
- **Jurisdiction:** locations carry recording_mode; transcript_only locations store no audio; paused locations are never dialled. UK-only launch: reject non +44 numbers at import for now.
- **Data boundaries:** scores attach to locations, never to named staff; no UI or export may invite naming an employee. Audio deletion at retention is real deletion.
- **Caller conduct:** scenario safety block (never request personal data, deflect AI questions once then exit politely, 4 minute cap) is injected server side into every agent, not left to per-scenario editing.

If a safety rule and a feature ever conflict, the safety rule wins and you ask us. An accidental call to an unauthorised business is the single worst thing this platform can do.

SECTION 9

Build order: seven milestones

Build strictly in this order. Each milestone ends with a demo to us and the listed proof. Do not start the next milestone before the demo.

Milestone	Scope	Proof of done
M1. Foundations (week 1)	Repo, Next.js app, Supabase schema and RLS, auth, org onboarding with attestation gate, seed script with the demo org	We log in, see the seeded league table, and a test proves no call record can exist for an unattested org
M2. Dashboard (weeks 2-3)	League table, location detail, call detail with audio seeking, scorecards, scenarios with AI generation, locations manager, team roles	Listen at 1:42 on the seeded Croydon call seeks the player; a coach role cannot edit a scorecard
M3. Calling (week 4)	Voice pool, agent_profiles sync to ElevenLabs, scheduler, outbound calls, intake webhook, allowlist and kill switch	A scheduled call rings our own phone, appears in the app with audio and transcript; a call to a non-allowlisted number is refused and logged
M4. Scoring (week 5)	Grading service, confidence flagging, review queue, weekly aggregates, alerts	The M3 call is auto-scored with evidence; a low-confidence fixture lands in the review queue; correcting it updates the customer view
M5. Billing (week 6)	Stripe products, Checkout, Portal, webhooks, gating, location-cap enforcement, setup fee	The full test-mode lifecycle in section 7 passes
M6. Admin (week 7)	All admin screens, impersonation, audit log, voice analytics	An operator regrades a call; every admin action appears in the audit log
M7. Polish and launch (week 8)	Email digests, empty states, error states, mobile pass, Lighthouse 90+, pen-test basics, backup and restore drill, runbook doc	We onboard the first real pilot org end to end without touching the database by hand

Working rules

- Small pull requests with clear descriptions. Anything ambiguous: ask before building, a five minute question beats a two day rebuild.
- Every external key lives in environment variables, documented in .env.example: SUPABASE_URL, SUPABASE keys, ELEVENLABS_API_KEY, ELEVENLABS_PHONE_NUMBER_ID, ANTHROPIC_API_KEY, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, RESEND_API_KEY, CALL_ALLOWLIST, ADMIN_EMAIL_ALLOWLIST.
- Tests required on: RLS policies, the attestation gate, the allowlist, scorecard version integrity, Stripe webhook handling, grader JSON parsing with malformed input.
- UI copy: British English, no em dashes ever, plain sentences. When unsure, copy the tone of the landing page.
- Never invent demo data beyond the canonical set (Richmond 94, Clapham 91, Battersea 87, Ealing 52, Croydon 38, Sutton 0, the Croydon evidence at 1:42). Consistency across every surface is deliberate.

Questions, access to the ElevenLabs and Stripe accounts, and milestone demos all go through the founder. Welcome aboard. Build the boring parts beautifully.